

碳化硅外延层厚度的确定

摘要

碳化硅是一种应用十分广泛的半导体材料，其外延层的厚度是影响性能及应用场景的重要性因素。本文主要针对其主要测量方法——红外干涉法进行了深入探究并对该方法提出了创新型的改进思路。

对于问题一，本文将此模型类比成薄膜干涉模型，根据两反射光的光程差公式 $\Delta = 2d\sqrt{n_1^2 - n_0^2 \sin^2 \varphi} + \frac{\lambda}{2}$ 分别对产生明、暗干涉条纹两种情况进行分析，通过常规物理量的公式推导法得到碳化硅外延层厚度 d 关于红外光谱波长 λ 、外延层折射率 n_1 以及红外光入射角 φ 的方程，即 $d = \frac{(n-1)\lambda}{4\sqrt{n_1^2 - \sin^2 \varphi}}$ ， $n \in N$ ，由此便可得出碳化硅外延层厚度的数学模型。

对于问题二，首先建立碳化硅外延层折射率简化模型，用待定系数法设出外延层折射率 n_1 中求解困难的量。再对问题一建立的方程模型用做差消元思想得到相对级数的方程。为了附录 1 中数据代入方程后使得方程尽可能成立，本文采用**最小二乘法**建立优化模型，通过调整多初始点遍历和调整决策变量的权重，得到最优外延层厚度为 $d_1 = 10.3159\mu m$ 。对附录 2 的数据的计算同附录 1，得出 $d_2 = 10.3871\mu m$ ，对结果的可靠性进行交叉验证。最后通过**偏导分析法**进行灵敏度检验，分析检验结果得到决策变量 d 的鲁棒性最弱，符合优化算法中给予 d 的高约束条件，进一步增加了结果的可靠性。

对于问题三，根据光的干涉条件，逐一分析产生多光束干涉的影响因素 (如光束来源、振动方向以及相位差) 最终确定要产生多光束满足的条件为 $\Delta = \frac{n\lambda}{2}$ ， $n \in N$ 。首先初步猜测考虑多光束干涉使外延层厚度计算精度提高。判断附件 3 和附件 4 的硅晶圆片出现了多光束干涉。重新建立精度更高的反射率模型，再运用物理光学相关的知识得到考虑多光束干涉下的反射率 R 的计算公式。建立优化模型使反射率尽量拟合实际数据，决策变量分别为 n_2 、 d 、 N 以及 m 。借助**模拟退火算法 (SA)** 得到硅外延层厚度的结果为 $d_3 = 4.8439\mu m$ ， $d_4 = 4.7974\mu m$ 。综上分析判断多光束干涉也会出现碳化硅晶圆片的测量结果中，为了降低碳化硅外延层计算的精度受其影响。建立了可以减少多光束干涉影响的线性拟合模型，近似认为波数与级数呈线性关系。用线性拟合的方法得到消除影响后的计算结果为 $d = 10.3057\mu m$ 。

最后，本文对碳化硅外延层厚度的确定模型得以改进，通过灵敏度分析验证改进后模型的合理性。这一改进为红外干涉法提供了合理的改进方案。同时由于本文中所用的模型提升了碳化硅外延层厚度的测量精度，在一定程度上也可以保障碳化硅材料器件性能的可靠性以及稳定性，让碳化硅材料器件实现由实验生产到工业应用的重大突破。

关键词： SiC(碳化硅) 多光束干涉 外延层厚度 最小二乘法 红外干涉法

一、问题重述

1.1 问题背景

碳化硅 (SiC) 是一种新兴的半导体材料, 其综合性能的优越表现使得其目前受到越来越多的关注。而碳化硅外延层厚度对外延材料相关器件的性能有着重大的影响。故如何制定一套科学、可靠、准确的碳化硅外延层厚度测试标准, 是人类推进外延材料器件发展工作的关键前提。

目前主流的无损伤测量方法主要是红外干涉法。因碳化硅外延层与衬底掺杂载流子浓度不同, 使得其拥有不同的折射率。而当红外光射入到外延层之后, 一部分光从外延层表面反射出来, 另外一部分光射入到外延层中在衬底表面反射回去, 这两束光在一定的条件下会产生干涉条纹。则研究人员可以通过红外光谱的波长、外延层的折射率以及红外光的入射角等参数来确定外延层的厚度。

1.2 问题要求

问题 1

若外延层与衬底界面之间只存在一次反射、透射产生干涉条纹, 请建立合适的模型求解出外延层厚度。

问题 2

根据问题 1 所得出的数学模型, 设计出合理的算法以确定外延层厚度。对附件 1 与附件 2 所提供的碳化硅晶圆片的光谱实测数据, 给出外延层厚度的计算结果, 并分析该结果的可靠性。

问题 3

若光波可以在外延层界面与衬底界面之间发生多次的反射与透射, 从而产生多光束干涉。请推导出产生多光束干涉的必要条件, 以及多光束干涉对外延层厚度计算精度可能产生的影响。

再根据多光束干涉的必要条件, 分析附件 3 与附件 4 所提供的硅晶圆片的测试结果是否出现多光束干涉, 并同问题 1、2 给出外延层厚度计算的数学模型与相应算法, 以及相应的计算结果。

若多光束干涉也会出现再碳化硅晶片的测试结果中, 即附件 1 与附件 2 中考虑多光束干涉。且这种情况会影响到碳化硅外延层厚度的计算精度, 请设法消除该影响, 并给出消除影响之后的计算结果。

二、问题分析

2.1 问题一分析

对于**问题一**，根据题目所给情景可以分析出当红外光在外延层界面与衬底界面只有一次反射、透射时，满足典型的**薄膜干涉模型**。因此本文将此情形认为是标准的薄膜干涉，并依据光程差公式通过递推关系求得两个方程，对该方程组进行化简即可确定所需的外延层厚度方程。

2.2 问题二分析

对于问题二，由于问题一中得出的外延层厚度方程中的外延层折射率 n_1 不是常数，故本文先对外延层折射率模型进行建立。通过建立外延层折射率 n_1 与红外光谱波长 λ 之间的参数方程将复杂的折射率计算公式简化。然后通过波峰 (波谷) 的外延层厚度方程，得出第 k 级与第 $k + 1$ 级波峰的函数关系，通过**消元法**将级数消去得到目标函数的子函数。接着类似地根据题中所给的原始数据求出折射率相对稳定波长区间相邻两波峰之间的函数关系，最后对这些子函数进行求平方和后得到最终优化函数，求得当目标函数最小时相应的决策变量的最优值，随后进行灵敏度分析以分析所得结果的合理性。

2.3 问题三分析

对于问题三，考虑多次反射和透射，产生了多光束干涉。由两束光的干涉条件推出特殊情景下多光束干涉的必要条件。初步分析外延层厚度的计算精度受多光束干涉的影响导致变小。进一步地，由附件数据，判断硅晶圆片会发生多光束干涉。通过光的反射率计算公式，分别建立新的外延层折射率模型和单目标优化模型。通过模拟退火算法 (SA) 得到相应的计算结果。观测附件并计算判断多光束干涉也会影响碳化硅晶圆片外延层厚度的计算精度。通过建立线性拟合模型并求解来设法消除其影响，最后得到消除影响后的外延层厚度。

三、模型假设

为简化问题，本文做出以下假设：

- 1. 假设研究此问题时，碳化硅的表面是平整光滑的平面。
- 2. 假设碳化硅外延层与衬底的掺杂是均匀的。
- 3. 假设在测量外延层厚度时，碳化硅体系一直处于常温常压下。
- 4. 假设在分析问题时不考虑光的吸收与散射。
- 5. 假设碳化硅晶体为单晶。

四、符号说明

符号	说明	单位
φ	光线从空气中进入外延层的入射角	rad
θ	光线从空气中进入外延层的折射角	rad
d	外延层厚度	μm
n_0	空气层的折射率	
n_1	外延层的折射率	
n_2	衬底的折射率	
k	级数	
λ	波长	μm
ν	波数	cm^{-1}
Δ	相邻两光线的光程差	μm
c	光速	m/s

五、问题一的模型的建立和求解

5.1 模型建立

本题主要是用红外干涉法来测量碳化硅晶体外延层厚度。而本文在模型假设中提到，外延层是一个平整光滑的平面，且外延层的厚度均匀，这样的假设符合红外干涉法的使用要求，故本文这部分的假设是合理的。

此时对于碳化硅的晶格方面带来的影响可以暂时被忽略掉，此时红外干涉法测量碳化硅外延层厚度就可以看成是一个标准的薄膜干涉模型。

接着根据题中所给信息结合薄膜干涉模型可以得出如下示意图：

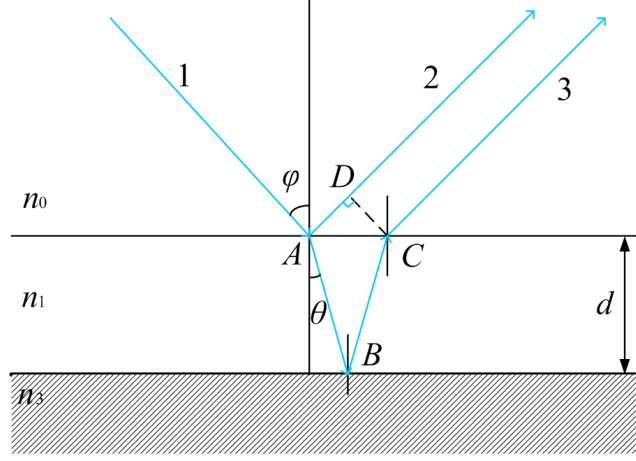


图 1 外延层厚度测量原理示意图

图 1 中上层部分为空气介质，其折射率为 $n_0 \approx 1$ ，而在本文中取 $n_0 = 1$ ；中层部分为外延层介质，其折射率为 n_1 (其中 n_1 不是常数，且与掺杂载流子浓度、红外光谱波长等因素相关)；下层部分为衬底介质，其折射率为 n_2 。

在上图中，光线 1 在空气介质中从某一点光源射出，射在外延层表面的点 A 上，入射角为 φ 。随后一部分反射为光线 2，其余部分则以折射角 θ 透射到衬底层表面的点 B 上。其中部分透射进衬底层中，剩余部分被反射到点 C 后折射出去 (此处应题目要求不考虑多光束干涉)，令这部分为光线 3。又因为假设中假定外延层介质中的掺杂载流子分布均匀，即对于某种确定的情况外延层 (衬底) 部分折射率处处相等，所以根据光的折射与反射的性质可以得出光线 2 与光线 3 是平行的。

过光线 3 从外延层介质折射入空气介质中的折射点 C 作光线 2 的垂线并与光线 2 相交于 D 点。要计算光线 2 与光线 3 的光程差，则计算光线 3 在折射率为 n_1 的外延层介质中的光程 $n_1(AB + BC)$ 与光线 2 在折射率为 n_0 的空气介质中的光程 $n_0 AD$ 的差值，即：

$$\Delta' = n_1(AB + BC) - n_0 AD \quad (1)$$

假设外延层的厚度为 d ，由图 1 可得：

$$\begin{cases} AB = BC = \frac{d}{\cos \theta} \\ AD = AC \sin \varphi = 2d \tan \theta \sin \varphi \end{cases} \quad (2)$$

将式 (2) 中的两个等式带入带式 (1) 中可以得出：

$$\Delta' = \frac{2d}{\cos \theta} (n_1 - n_0 \sin \theta \sin \varphi) \quad (3)$$

根据折射定律 $n_0 \sin \varphi = n_1 \sin \theta$ ，上式可以改写成如下表达：

$$\Delta' = 2d \sqrt{n_1^2 - n_0^2 \sin^2 \varphi} \quad (4)$$

此外, 因为在这个薄膜干涉模型中, 红外光是从光疏介质射入光密介质, 即 $n_1 > n_0$, 则还需要考虑到相位跃变与附加相移。

又因为在该薄膜干涉模型中, 附加相移带来的影响相对相位跃变十分复杂, 故在本文的模型中仅考虑相位跃变带来的影响, 即半波损失为 $\frac{\lambda}{2}$ 的情况

则两反射光的总光程差为:

$$\Delta = 2d\sqrt{n_1^2 - n_0^2 \sin^2 \varphi} + \frac{\lambda}{2} \quad (5)$$

5.2 模型求解与结果

因为空气介质中的折射率 $n_0 = 1$, 根据相干波的干涉条件可以得出:

$$\begin{cases} \text{明条纹:} & 2d\sqrt{n_1^2 - \sin^2 \varphi} + \frac{\lambda}{2} = k\lambda \\ \text{暗条纹:} & 2d\sqrt{n_1^2 - \sin^2 \varphi} + \frac{\lambda}{2} = (k + \frac{1}{2})\lambda \end{cases} \quad (6)$$

左右两边移项可以得出关于外延层厚度 d 的关系式:

$$\begin{cases} \text{明条纹:} & d = \frac{k\lambda - \frac{\lambda}{2}}{2\sqrt{n_1^2 - \sin^2 \varphi}} \\ \text{暗条纹:} & d = \frac{(k + \frac{1}{2})\lambda - \frac{\lambda}{2}}{2\sqrt{n_1^2 - \sin^2 \varphi}} \end{cases}$$

其中 $k \in N$ 为级数, λ 为红外光谱的波长, φ 为红外光的入射角, n_1 为外延层的折射率, 且外延层折射率 n_1 不是常数, 它的大小与掺杂载流子浓度、红外光谱波长等参数有关。

将明、暗条纹的表达式整合根据周期性到一起, 得:

$$d = \frac{(n - 1)\lambda}{4\sqrt{n_1^2 - \sin^2 \varphi}} \quad (7)$$

至此问题 1 得以解决。

六、问题二的模型的建立和求解

6.1 模型建立

6.1.1 外延层折射率简化模型

碳化硅 (SiC) 属于六方晶系半导体材料, 从电磁学角度分析, 其折射率本质是光在材料中传播时“介电响应”的宏观体现, 所以满足:

$$n = \sqrt{\varepsilon_r \cdot \mu_r} \quad (8)$$

其中 n 为折射率, ε_r 为相对介电常数, μ_r 为相对磁导率, 对于非磁性材料 SiC , $\mu_r \approx 1$ 。

因此上式可以化简成: $n = \sqrt{\varepsilon_r}$ 。

对于相对介电常数 ε_r , 由于本文中所提及的波长范围为红外区域, 此时介电影响主要是自由电子在主导, 故应采用 *Drude* 模型进行分析^[2]。

有:

$$\begin{cases} \varepsilon_r = \varepsilon_\infty - \frac{\omega_p^2}{\omega^2 + i\omega\gamma} \\ \omega = \frac{2\pi c}{\lambda} \\ \omega_p^2 = \frac{Ne^2}{\varepsilon_0 m^*} \\ \gamma = \frac{e}{m^*} \end{cases} \quad (9)$$

其中 c 为光速, ω 为电磁波角频率, ε_0 为真空介电常数, ε_∞ 为高频介电常数, N 为自由载流子浓度, e 为电子电荷量, m^* 为电子的有效质量。

由上述分析可以对外延层的折射率进行简化, 简化后得到:

$$n_1 = \sqrt{a - b\lambda^2} \quad (10)$$

其中, a 、 b 为参数, 并假定自由载流子浓度为已知参数。

6.1.2 目标函数的建立

对第 k 级波峰, 其折射率满足: $2d\sqrt{n_{1k}^2 - \sin^2 \varphi} = k\lambda_k$, 同理可得第 $k+1$ 级波峰满足的式子, 联立后得到:

$$\begin{cases} 2d\sqrt{n_{1k}^2 - \sin^2 \varphi} + \frac{\lambda_k}{2} = k\lambda_k \\ 2d\sqrt{n_{1(k+1)}^2 - \sin^2 \varphi} + \frac{\lambda_{k+1}}{2} = (k+1)\lambda_{k+1} \end{cases} \quad (11)$$

由于级数的具体值不清楚, 而级数之前的差值即相对关系已知, 故对式 (11) 中两个方程左右两边同时除以 λ 之后再用下式减上式可以将级数 k 消去, 即:

$$\frac{2d}{\lambda_{k+1}}\sqrt{n_{1(k+1)}^2 - \sin^2 \varphi} - \frac{2d}{\lambda_k}\sqrt{n_{1k}^2 - \sin^2 \varphi} - 1 = 0 \quad (12)$$

因为去掉异常值的附录中的数据无法保证其级数等于整数, 由此用最小二乘法的思路让上述式子的平方尽可能接近 0 为目标建立单优化模型

故令等式左边为 f_k , 得到目标函数的一个子函数。

将附件 1 的数据进行平滑等处理, 并剔除异常值后, 绘制成折线图如下图 2 所示:

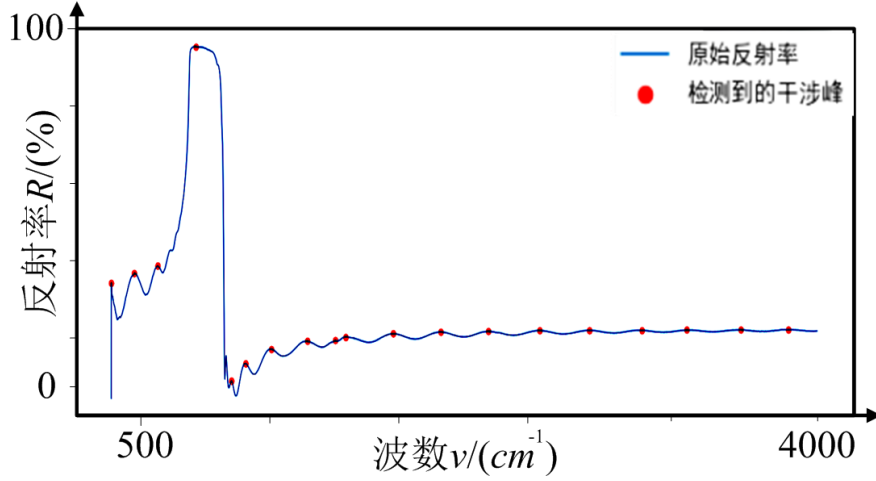


图 2 附件 1 数据可视化

因为当波数相对较小时，反射率的变化十分明显，且还存在数值突跃，很显然这部分的数据是不可靠的，猜测这部分发生的原因是测量时某些不可抗外界因素对测量结果产生的系统误差。

根据图 2，对反射率变化较为平稳时的波数区间内相邻两波峰数据进行上述处理，可以得出目标函数的每个子函数，对子函数求平方和之后便能得出此问所需优化的目标函数。

由于级数 k 的取值为 $k \in (8, 15)$ ，故最终目标函数 P 为：

$$\sum_{k=8}^{15} \left(\frac{2d}{\lambda_{k+1}} \sqrt{n_{1(k+1)}^2 - \sin^2 \varphi} - \frac{2d}{\lambda_k} \sqrt{n_{1k}^2 - \sin^2 \varphi} - 8 \right)^2 \quad (13)$$

综上，此问可以看成是一个决策变量为 a 、 b 、 d 的单目标多决策变量优化模型，即：

$$\begin{aligned} & \min P \\ & \begin{cases} n_{1k} > 1 \\ d > 0 \\ \lambda_k = \frac{1}{\nu} \\ \nu \in [400, 4000] \\ \varphi \in [0, \frac{\pi}{2}] \end{cases} \end{aligned} \quad (14)$$

其中需要格外注意的一点是此处 $d > 0$ 表示的是外延层厚度不过于薄，即 d 不是严格意义上的大于 0。这缘于红外干涉的根本原理，影响其测量外延层厚度的对主要因素便是红外反射光谱中干涉条纹的周期，厚度的减小会使得周期相应的增大，从而使得在某一波长范围内拥有的干涉条纹周期数减少，增加了测量的困难也加大了误差。

6.2 模型求解

Step1: 根据图 2 中数据点初始设定波峰的合理区间，超出该区间的波峰因误差过大不予考虑。

Step2: 对通过 **Step1** 的波峰进行处理后计算出相邻峰波长差。

Step3: 代入目标函数中，计算理论值与实际值的平方误差。

Step4: 根据物理量的实际意义对其进行目标函数的取值进行一定的约束。

Step5: 通过多初始点遍历提升该非线性优化的全局最优性。对每一组初始点，采用 **SLSQP** 优化方法来执行优化。其中对三个决策变量赋予不同的权重，其中主要权重给到决策变量 d ，该权重可动态调整，目的使得最终结果满足物理意义的约束。最终输出结果。

6.3 求解结果

通过上述求解方式，本文用 *Python* 对附件 1 进行处理，最终得出结果——最优外延层厚度为 $d_1 = 10.3159\mu m$ 。

又因为附件 2 与附件 1 的信息基本相同，故对附件 2 中的数据同上处理也可得到附件 2 的优化模型，同理得出——最优外延层厚度为 $d_2 = 10.3871\mu m$

本文将附件 1 与附件 2 求得的结果进行对比，即可初步验证该模型算法的合理性。

因为附件 1 求出的最优外延层厚度与附件 2 求出的最优外延层厚度之差的绝对值为 $0.0712\mu m$ ，相差值小于 $1e-5cm$ ，符合预期，故初步认为本文的算法模型是合理的。

6.4 模型的灵敏度检验

对于最终目标函数 P 其决策变量为 a 、 b 与 d ，此处的灵敏度分析本文采用偏导数法进行分析。

对于因变量 P ，分别对三个决策变量求偏导。则偏导数的值便可反映自变量对因变量的灵敏度。

通过 *Python* 对附件 1 与附件 2 进行处理，可以求出该条件下参数的灵敏度值如下：

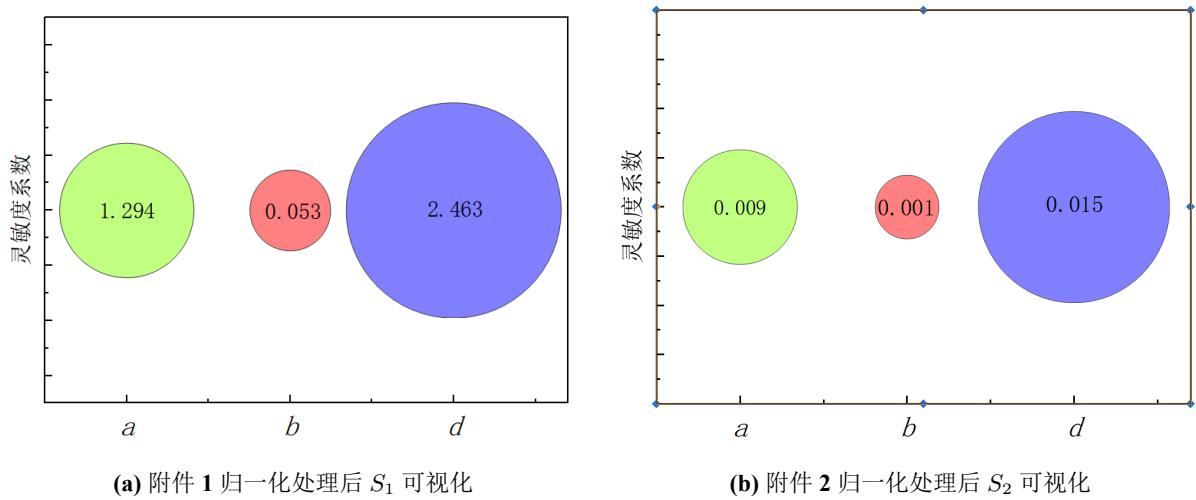
$$\left\{ \begin{array}{l} \frac{\partial P}{\partial a_1} = 7.8193 \times 10^{-3} \\ \frac{\partial P}{\partial a_2} = 1.7894 \times 10^{-4} \\ \frac{\partial P}{\partial b_1} = -2.6787 \times 10^{-1} \\ \frac{\partial P}{\partial b_2} = -2.1061 \times 10^{-2} \\ \frac{\partial P}{\partial d_1} = 5.7474 \times 10^{-3} \\ \frac{\partial P}{\partial d_2} = 1.1137 \times 10^{-4} \end{array} \right.$$

又因为最终目标函数 P 与其决策变量的量纲不同，此时单纯地比较三者的偏导数取比较灵敏度就不太合理了。因此我们再对其做归一化处理，对其相对灵敏度 S 进行比较即可。即：

$$\begin{cases} S_{a_i} = \frac{a_i}{P} \cdot \frac{\partial P}{\partial a} \\ S_{b_i} = \frac{b_i}{P} \cdot \frac{\partial P}{\partial b} \\ S_{d_i} = \frac{d_i}{P} \cdot \frac{\partial P}{\partial d} \end{cases} \quad (15)$$

其中相对灵敏度表示自变量变化 1% 时，因变量变化的百分比。

将结果可视化后得到如下图所示：



由图中信息可以推断出在三个决策变量中 d 是最敏感的参数需要精确控制；而 b 的灵敏度却异常的低，这说明对于这个优化模型来说参数 b 的影响几乎可以忽略不计。

综上所述，在优化过程中，应该首先保证参数 d 的精度，为此可以适当放宽对参数 b 的约束。而这也恰好符合了优化中对参数 d 的约束权重最大这一处理方式，故可证明该结果的可靠性。

七、问题三的模型的建立和求解

7.1 外延层厚度 d 的模型建立

在对外延层厚度的测量过程中，实际情况可能不是那么理想，如红外光很可能会有在外延层中发生多次反射与折射。这样便会在一定程度上影响我们模型的结果。

因此，本文将模型做拓展，即考虑入射光在外延层内多次反射与折射，产生多光束干涉。

首先，需要明确能够产生多光束干涉的必要条件。

7.1.1 产生多光束干涉必要条件模型的建立

(一)、多光束干涉条件的推导

因为在薄膜干涉中，要想产生多光束干涉可以从以下角度进行分析讨论：

(1) 从光束来源角度分析：由于不同频率的光在薄膜干涉中会各自形成一套多光束干涉条纹，且每套条纹的条纹间距也各不相同，此时干涉条纹便会叠加，从而使得无法观测到清晰的多光束干涉现象。由此，**入射光必须为单色光**，而本文中考虑的情况下入射光均为红外光，属于单色光，即在本文考虑的情况下，并不能作为产生多光束干涉的必要条件。

(2) 从振动方向角度分析：由于光波产生干涉其实就是多束光在空间某点的振动的矢量叠加。因此要产生明显的干涉条纹，就需要满足让振动的矢量叠加最大化，即**这束光的振动方向必须一致**。因为本文中的入射光属于红外光，故在本文考虑的情况下，振动方向都是稳定不变的。故振动方向在本文考虑的情况中并不能作为产生多束光干涉的必要条件。

(3) 从相位差角度分析：

i 空间相干性因为本文使用的光源为点光源，故空间相干性很高，不需要进行过多的考虑。

ii 时间相干性要使得相位差是稳定的，那么入射光的相干长度需大于多光束的光程差。即光程差需要满足以下条件：

$$\Delta = 2d\sqrt{n_1^2 - \sin^2 \varphi} + \frac{\lambda}{2} = \begin{cases} k\lambda, & k = 1, 2, \dots (\text{增强}) \\ (k + \frac{1}{2})\lambda, & k = 0, 1, 2, \dots (\text{减弱}) \end{cases} \quad (16)$$

综上所述，虽然对于产生多光束干涉需要考虑很多因素，但在本题的背景下，只需要满足式 (16) 即可。

(二)、对计算精度可能产生的影响

考虑多光束干涉会使反射率变大，更接近真实值，对于特定反射峰值或谷值考虑多光束干涉会使外延层厚度计算精度显著提升。

7.1.2 反射率模型的建立

根据光的电磁理论可知——光是一种特定频率范围内的电磁波。故光在介质表面的折射与反射实际上就是光与介质的相互作用。因此，单色光下各向同性介质中的麦克斯

韦方程组解的电场、磁场表示如下：

$$\begin{cases} \vec{E}_n = \vec{E}_0 e^{i(\vec{k}_n \cdot \vec{r} - \omega_n t)} \\ \vec{H}_n = \vec{H}_0 e^{i(\vec{k}_n \cdot \vec{r} - \omega_n t)} \\ \vec{H}_0 = \frac{1}{\omega \mu} \vec{k} \times \vec{E}_0 \\ n = i, t, r \end{cases} \quad (17)$$

其中 $\vec{k}$ 为波矢量，大小为 $k = \frac{\omega n_1}{c}$ ，角标 i, t, r 分别表示入射光、折射光和反射光， $\vec{r}$ 表示任意一点的矢径。

现以入射光第一次射入介质中时的入射点 O 为原点，平视光路图沿着界面向右为 x 轴，垂直 x 轴向外为 y 轴， z 轴与界面 xOy 垂直向上且过点 O 建立空间直角坐标系。

以空气介质与外延层介质为例，有：

$$\begin{cases} n_0 \sin \theta_i = n_1 \sin \theta_t \\ \theta_i = \theta_r \end{cases} \quad (18)$$

根据式 (17) 对 S 偏振光进行分析，即电场沿 y 轴方向时：

(1) 对于入射光：

$$\begin{cases} A_i = e^{i(k_i x \sin \theta_i - k_i z \cos \theta_i - \omega t)} \\ \vec{E}_i = E_{i0} \hat{y} A_i \\ \vec{H}_i = \frac{n_0}{\mu_0 c} E_{i0} (\cos \theta_i \hat{x} + \sin \theta_i \hat{z}) A_i \end{cases} \quad (19)$$

(2) 对于反射光：

$$\begin{cases} A_r = e^{i(k_r x \sin \theta_r - k_r z \cos \theta_r - \omega t)} \\ \vec{E}_r = E_{r0} \hat{y} A_r \\ \vec{H}_r = \frac{n_0}{\mu_0 c} E_{r0} (-\cos \theta_r \hat{x} + \sin \theta_r \hat{z}) A_r \end{cases} \quad (20)$$

(3) 对于折射光：

$$\begin{cases} A_t = e^{i(k_t x \sin \theta_t - k_t z \cos \theta_t - \omega t)} \\ \vec{E}_t = E_{t0} \hat{y} \\ \vec{H}_t = \frac{n_1}{\mu_0 c} E_{t0} (\cos \theta_t \hat{x} + \sin \theta_t \hat{z}) A_t \end{cases} \quad (21)$$

其中 $\hat{x}$ 、 $\hat{y}$ 、 $\hat{z}$ 分别表示沿 x 、 y 、 z 轴方向的单位向量。

因为在空气与外延层介质处的电场与磁场的切向分量是连续的，故有：

$$E_{i0} + E_{r0} = E_{t0} \quad (22)$$

且

$$n_0 \cos \theta_i (E_{i0} - E_{r0}) = n_1 \cos \theta_t E_{t0} \quad (23)$$

综上联立式 (22) 与式 (23)，得：

$$n_1 \cos \theta_t (E_{i0} + E_{r0}) = n_0 \cos \theta_i (E_{i0} - E_{r0}) \quad (24)$$

则处理上式可得到：

$$\begin{cases} r_s = \frac{E_{r0}}{E_{i0}} = \frac{n_0 \cos \theta_i - n_1 \cos \theta_t}{n_0 \cos \theta_i + n_1 \cos \theta_t} \\ t_s = \frac{E_{t0}}{E_{i0}} = \frac{2n_0 \cos \theta_i}{n_0 \cos \theta_i + n_1 \cos \theta_t} \end{cases} \quad (25)$$

通过更换介质折射率 n 可以求得任意两界面的 S 偏振振幅反射系数与 S 偏振振幅透射系数。

同理，根据式 (17) 对 P 偏振光进行分析，即电场与 y 轴垂直时：

(1) 对于入射光：

$$\begin{cases} A_i = e^{i(k_i x \sin \theta_i - k_i z \cos \theta_i - \omega t)} \\ \vec{E}_i = E_{i0}(-\sin \theta_i \hat{x} + \cos \theta_i \hat{z}) A_i \\ \vec{H}_i = \frac{n_0}{\mu_0 c} E_{i0} \hat{y} A_i \end{cases} \quad (26)$$

(2) 对于反射光：

$$\begin{cases} A_r = e^{i(k_r x \sin \theta_r - k_r z \cos \theta_r - \omega t)} \\ \vec{E}_r = E_{r0}(-\sin \theta_r \hat{x} - \cos \theta_r \hat{z}) A_r \\ \vec{H}_r = \frac{n_0}{\mu_0 c} E_{r0} \hat{y} A_r \end{cases} \quad (27)$$

(3) 对于折射光：

$$\begin{cases} A_t = e^{i(k_t x \sin \theta_t - k_t z \cos \theta_t - \omega t)} \\ \vec{E}_t = E_{t0}(-\sin \theta_t \hat{x} + \cos \theta_t \hat{z}) A_t \\ \vec{H}_t = \frac{n_1}{\mu_0 c} E_{t0} \hat{y} \end{cases} \quad (28)$$

由电场与磁场切向是连续的，有：

$$\sin \theta_i (E_{i0} + E_{r0}) = \sin \theta_t E_{t0} \quad (29)$$

且

$$n_0 (E_{i0} + E_{r0}) = n_1 E_{t0} \quad (30)$$

综上联立式 (29) 与式 (30)，得：

$$E_{t0} = \frac{n_0}{n_1} (E_{i0} + E_{r0}) \quad (31)$$

用折射定理做替换后可以得到：

$$\begin{cases} r_p = \frac{E_{r0}}{E_{i0}} = \frac{n_1 \cos \theta_i - n_0 \cos \theta_t}{n_1 \cos \theta_i + n_0 \cos \theta_t} \\ t_p = \frac{E_{t0}}{E_{i0}} = \frac{2n_0 \cos \theta_i}{n_1 \cos \theta_i + n_0 \cos \theta_t} \end{cases} \quad (32)$$

根据上述求出的 r_s 、 t_s 、 r_p 、 t_p 即是著名的菲涅尔系数^[3]。通常来说，若是已知界面两侧的折射率与折射角，就可以通过菲涅尔定律求解折射角、折射系数与反射系数等量。

将上述讨论的单一反射、透射的菲涅尔系数进一步深入探究，即考虑外延层发生多次反射、透射的情况。

依据问题三中的描述得到下示多光束干涉示意图：

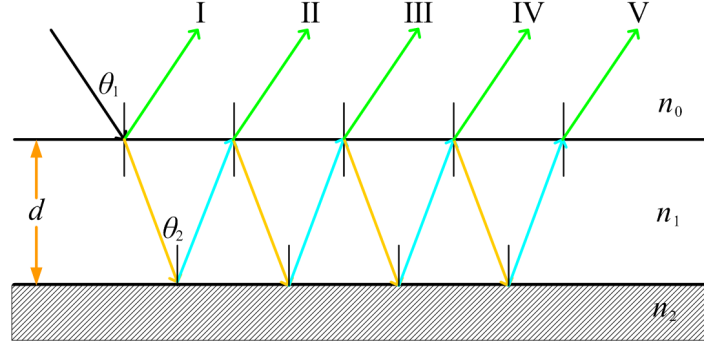


图 4 多光束干涉示意图

假设由上层空气层射入中间层外延层的透射率为 t ，从外延层射入空气层的透射率为 t' ，在空气层与外延层反射的反射率为 r_1 ，在外延层与下层衬底层反射的反射率为 r_2 。则可以依此得到光束 I-V 的反射光线 (绿色部分) 分别为^[4]： r_1 、 $tt'r_2$ 、 $-tt'r_1r_2^2$ 、 $tt'r_1^2r_2^3$ 、 $-tt'r_1^3r_2^4$ 。

若不考虑第一项，从第二项开始看可以发现，每后一项都是前一项的 $-r_1r_2$ 倍。故可以将其看成是一个从第二项开始的无穷等比数列。

由式 (17) 可知 I-V 各个光束的反射光线的振幅分别为： r_1 、 $tt'r_2e^{i\delta}$ 、 $-tt'r_1r_2^2e^{2i\delta}$ 、 $tt'r_1^2r_2^3e^{3i\delta}$ 、 $-tt'r_1^3r_2^4e^{4i\delta}$ 。故可以求得合成的复振幅为：

$$r = r_1 + \frac{tt'r_2e^{i\delta}}{1 + r_1r_2e^{i\delta}} \quad (33)$$

根据透射光强的公式可以知道：

$$I = \frac{n}{2\mu_0 c} E_0^2 \quad (34)$$

又由能量守恒定理可知：

$$I_i = I_r + I_t \quad (35)$$

联立式 (34) 与式 (35) 可得到：

$$n_0 E_{i0}^2 = n_0 E_{r0}^2 + n_1 E_{t0}^2 \quad (36)$$

即：

$$\begin{cases} n_0 = n_0 r_1^2 + n_1 t^2 \\ n_1 = n_1 r_2^2 + n_0 (t')^2 \end{cases} \quad (37)$$

故，有：

$$tt' = n_1(1 - r_1^2) \quad (38)$$

此时，合成的复振幅可以化简为：

$$r = \frac{r_1 + (1 - n_1)r_1^2 r_2 e^{i\delta} + n_1 r_2 e^{i\delta}}{1 + r_1 r_2 e^{i\delta}} \quad (39)$$

其中 $\delta = \frac{4\pi}{\lambda} nd \cos \theta$ (θ 为折射角)^[4]。

综上所述，可以得出多光束干涉的反射率 R ，即：

$$R = r^2 = \left(\frac{r_1 + (1 - n_1)r_1^2 r_2 e^{i\delta} + n_1 r_2 e^{i\delta}}{1 + r_1 r_2 e^{i\delta}} \right)^2 \quad (40)$$

又有：

$$t = \frac{tt'}{1 + r_1 r_2 e^{i\delta}} \quad (41)$$

类似地可以得到透射率 T ，即：

$$T = t^2 = \left(\frac{tt'}{1 + r_1 r_2 e^{i\delta}} \right)^2 \quad (42)$$

7.1.3 优化模型的建立

该优化模型的本质与问题 2 中的优化类似。故在问题 2 的背景前提下，对本问的优化模型进行建立。又通过对处理过后的硅晶原片的测试结果拟合图像进行分析，发现明显有三角函数的周期波荡性质，故可以认为会出现多光束干涉。

由于问题 3 的背景是多光束干涉，此时，仅仅比较光程差来判断优化便不能够使得计算结果的可靠性，容易陷入到局部最优中。因此，本文针对反射率建立了一套计算模型，将原始数据进行平滑处理、剔除异常值之后所遍历数据点的理论反射率与实际反射率的差值平方和进行全局优化。

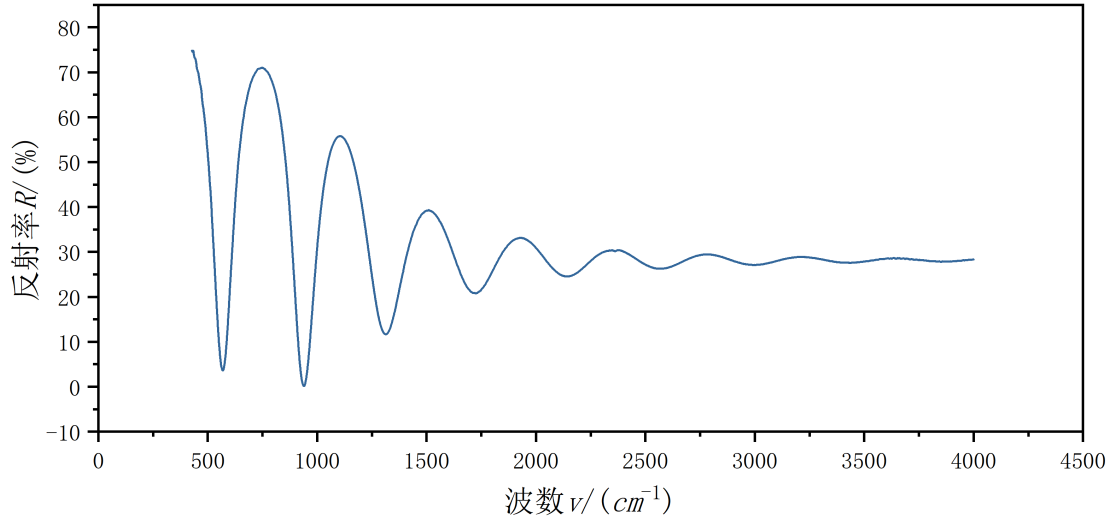


图 5 附件 3 数据可视化

结合上文所述可以得到最终目标函数 Q 为：

$$\sum_j (R - R')^2 \quad (43)$$

对于这个优化模型其决策变量为：衬底折射率 n_2 、外延层厚度 d 、自由载流子浓度 N 、电子有效质量 m^* 。

综上所述，此问可以看成决策变量为 n_2 、 d 、 N 、 m^* 的单目标多决策变量优化模型，即：

$$\begin{aligned} & \min Q \\ & \begin{cases} \theta_n \in [0, \frac{\pi}{2}], & n = i, r, t \\ \theta_i = \theta_r > \theta_t \\ n_1 > 1 \\ n_2 > 1 \end{cases} \end{aligned} \quad (44)$$

7.1.4 模型求解

Step1: 对预处理过的图 5 中的附件 3 数据可视化图像进行分析，选取相对较为平稳的波数区间。

Step2: 使用定步长遍历该平稳波束区间的数据。

Step3: 用最小二乘法计算目标函数算这些数据的理论值与实际值的平方误差。

Step4: 用双退火算法通过调整决策变量来使平方误差的和尽可能小，并输出使得平方误差最小时的外延层厚度 d 。

7.1.5 求解结果

通过上述分析思路, 本文先对附件 3 进行处理, 最终得出结果——最优外延层厚度为 $d_3 = 4.8439\mu m$ 。

同问题 2 中所分析的, 对附件 4 中的数据一样可以进行上述处理, 得到最优外延层厚度为 $d_4 = 4.7974\mu m$ 。

将附件 3 与附件 4 求得的结果进行对比分析, 即可对该模型的合理性进行验证。

因为附件 3 求出的最优外延层厚度附件 4 求出的最优外延层厚度之差的绝对值为 $0.0465\mu m$ 相差值小于 $1e-5cm$, 符合预期, 故可以初步认为本文的算法模型是合理的。

7.2 多光束干涉影响外延层厚度模型的计算

7.2.1 模型的准备^[5]

通过上述模型的求解, 不难发现, 对于多光束干涉情况来说需要实现全域拟合的拟合难度相当之大。故本文对此做出来一些新的简化, 使得拟合难度有所下降的前提下提升拟合程度且在一定程度上减小了多光束干涉对外延层厚度计算的影响。

由式 (5) 可知, 对于任意以波数 v , 都存在一个级数 K 与之对应。故二者满足一定的函数关系, 又对式 (5) 进行变换可以得到下式:

$$K = 2d|n_1 v \cos \theta_2| + \frac{1}{2} \quad (45)$$

由公式 $n_1 = \sqrt{\varepsilon_r}$ 以及对 ε_r 的分析中可知, 在 *Drude* 模型下外延层折射率 n_1 实际上是一个复数, 而这部分在前面的分析之中本文选择了忽略虚部来简化折射率 n_1 但在一定程度上增大了误差。

为了使得结果更加准确, 此处决定对前文中的折射率的确定进行优化, 即通过取绝对值得到该复数的模长, 以此进行计算。

因为 K 是级数, 通过附件所给的波数-反射率数据无法直接或间接地得到任何数据的级数。因此本文通过求级数差 ΔK 与波数 v 进行拟合处理。即:

$$\Delta K = 2d|n_1 v \cos \theta_2| + \frac{1}{2} - K_0 \quad (46)$$

其中上式中的 ΔK 是任意点与参考点之间的波峰与波谷数量和。

7.2.2 模型的建立

对式 (46), 拟合时其自变量为 ω 因变量为 ΔK , 进行拟合后预期得到一个类似于 $y = Ax + B$ 其中 A 、 B 中有部分是未知的。需要求这未知部分即根据拟合后的一次函数图像斜率与截距。但其中需要注意的是在对 K_0 分析时需知道 K_0 不连续且只能取到半整数。

后对附件 1 中的结果进行拟合得到在波数从 $1900(\text{cm}^{-1})$ 到 $3800(\text{cm}^{-1})$ 的线性拟合结果，如下图所示：

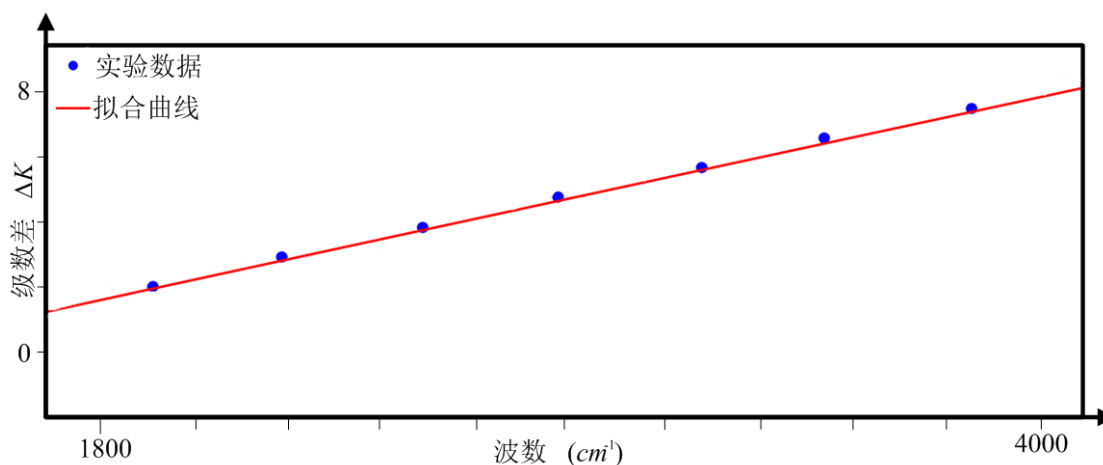


图 6 拟合曲线

由图可以看出拟合程度很高且由式 (46) 可知到拟合后该一次函数图像的斜率，即知道 $A = 2n_1 d \cos \theta_2$ 的值。而其中， $\cos \theta_2$ 可以看成是一个定值。 n_1 虽然是一个与波数 ω 相关的函数，但由于要确保拟合的线性，此处结合折射率的变化较小这一特点，也将其看成是一个定值。

此时即斜率是一个关于外延层厚度 d 的函数。此时，只需要假定拟合之前所选取参考点的相应数据，即可求出相应的最优厚度。

7.2.3 模型的求解

ΔK 理论上应为 0.5 的整数倍。由问题二计算的相对级数结果，可知算出的 ΔK 接近 0.5 的整数倍，这与分析结果相符。为了使拟合更精确，把 ΔK 近似成 0.5 的整数倍来提高计算精度。同问题二取后面反射率稳定后的波数确定参考点级数，进行拟合计算得到 $A = 0.000828$ 。由于没有实测数据对应的真实参数，故借助问题二中的外延层折射率简化模型计算结果，近似认为外延层折射率 $n_1 = 1.9473$ 。由光的折射定理 $\cos \theta_2 = \sqrt{1 - (\sin \theta_1 / n_1)^2}$ 确定 θ_2 ，最终计算得到外延层厚度 $d_1 = 10.0700\mu\text{m}$ 、 $d_2 = 10.5414\mu\text{m}$ 。故， $d = \frac{d_1 + d_2}{2} = 10.3057\mu\text{m}$

八、模型的评价

8.1 模型的优点

- 优点 1 做出了合理的简化，使得模型容易计算，更易理解运用。

- 优点 2 模型忽略了非主要的影响因素，把握主要因素。
- 优点 3 线性拟合的方法减少了复杂的计算而得到较为精确的结果。

8.2 模型的缺点

- 缺点 1 没有计算全域拟合模型，拟合出全部波数对应的图像，且忽略了某些重要参数对结果的影响。

8.3 模型的推广与改进

本文的模型对折射率，厚度等加以改变，能够运用到三维成像，薄膜厚度与折射率测量，材料应力等更多的领域。

参考文献

- [1] DeepSeek, DeepSeek-V3-0324, 深度求索 (DeepSeek), 2025-09-05
- [2] 爱分享的玲玲. 基于德鲁德-洛伦兹模型的介电常数参数拟合方法：从折射率 (n,k) 到光学响应分析 [Z].[知乎].2025
- [3] 吴咏桐, 丁永文. 菲涅尔公式的实验研究 [J/OL]. 大学物理实验.2019.DOI: 10.14139/j.cnki.cn22-1228.2019.03.012
- [4] 王荣堃. 薄膜式法布里珀罗光纤传感器的有限元分析及应用研究 [D]. 山东大学.2020
- [5] 李志云. 基于 FTIR 技术的 4H-SiC 同质外延材料的测试. 西安电子科技大学.2010

附录 A 文件列表

文件名	功能描述
code1.py	问题二程序代码
code2.py	问题三程序代码 1
code3.py	问题三程序代码 2

附录 B 代码

code1.py

```
1 # 本论文运行环境为Python，版本为 3.12.11
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.signal import find_peaks
6 from scipy.optimize import minimize, dual_annealing
7 import math
8 import cmath
9
10 def process_and_find_peaks(spectral_data_path,
11     min_height=0.6,
12     min_distance=50,
13     prominence=0.3
14 ):
15     """先读取附件中数据并进行干涉峰的动态检测"""
16     #读取Excel表格数据，同时将列名指定为波数("v")和反射率("R")
17     df = pd.read_excel(spectral_data_path, header=None, names
18         =["v", "R"])
19
20     # 旨在一条龙实现“类型转换→去缺失→排序→重置索引”的整合，
21     # 把数据类型转换
22     df[["v", "R"]] = df[["v", "R"]].apply(pd.to_numeric,
23         errors='coerce')
```

```

24     df = df.dropna(subset=['v', 'R'])
25
26     # 排序并把索引重新设置
27     df = df.sort_values('v').reset_index(drop=True)
28
29     # 添加波长列，同时在使用公式时顺便进行了单位的转换
30     df["λ"] = 1e4 / df["v"]
31     # 基于  $v[\text{cm}^{-1}] = 10^4/\lambda[\mu\text{m}]$  的光谱学定义，我们得到了波长转换
    公式：  $\lambda[\mu\text{m}] = 10^4/v[\text{cm}^{-1}]$ 
32
33     # 2. 动态检测干涉峰
34     # 设置参数可自动调整，直到找到合理数量的峰(预估5-30个为合
    理的可接受的范围)，随即终止
35     # 为了避免变量操作太过分散，我们采用封装存储策略，利用
    params字典以及adjust字典分别集中管理可调整参数和对应的调整因
    子
36     params = {'min_h': min_height, 'min_d': min_distance}
37     adjust = {'under': (0.9, 0.8), 'over': (1.1, 1.2)}
38     peaks_idx, properties = [], {}
39
40     for attempt in range(3):
41         peaks_idx, properties = find_peaks(
42             df["R"],
43             height = params['min_h'],
44             distance = params['min_d'],
45             prominence = prominence,
46             width = 20
47         )
48
49     # 检查峰数量是否是合理的，中断后可方便调参
50     cnt = len(peaks_idx)
51     if 5 <= cnt <= 30:
52         break
53
54     # 用元组解包实现参数调整,减少嵌套

```

```

55         h_factor, d_factor= adjust['under'] if cnt < 5 else
adjust['over']
56         params['min_h'] *= h_factor
57         params['min_d'] *= d_factor
58     else:
59         print("警告:找不到合理数量的干涉峰, 使用目前检测结果")
60
61     # loc可以做到批量提取峰数据, 通过values.T转置后直接解包赋值, 效率提升一大截
62     peaks_data = df.loc[peaks_idx, ['v', 'R', 'λ']].values.T
63     peaks_v, peaks_R, peaks_λ = peaks_data
64
65     # 计算相邻峰波长差
66     delta_λ = np.diff(peaks_λ)
67
68     # 打印相邻峰波长差
69     for idx, delta in enumerate(delta_λ, start=1):
70         print(
71             f"峰{idx}和峰{idx+1}的波长差: {delta:.4f} μm "
72             f"(λ□={peaks_λ[idx-1]:.4f} μm, λ□={peaks_λ[idx]:.4f} μm)"
73         )
74     return peaks_idx, peaks_v, peaks_R, peaks_λ, delta_λ, df
75
76 # 处理附件1(替换附件二同理)数据 (本论文已经优化了参数的管理方式, 还是用字典封装)
77 attach1_params = {
78     "spectral_data_path": "附件1.xlsx",
79     "min_height": 0.8,
80     "min_distance": 80,
81     "prominence": 0.3
82 }
83
84 #函数一次返回多个结果
85 (attach1_peak_idx, attach1_peak_v, attach1_peak_R,
86 attach1_peak_λ, attach1_delta_λ, attach1_df ) =

```

```

process_and_find_peaks(**attach1_params)
87
88 def optimize_parameters(peak_wavelengths, delta_λ, theta0=10):
89     """优化计算a,b,d参数(采用多初始点优化, 同时增强折射率约束)
    """
90     theta0_rad = math.radians(theta0)
91
92     # 待优化的目标函数1: 理论波长差公式
93     def objective1(params):
94         a, b, d = params
95         theta0_rad = math.radians(10) # 按题目要求入射角固定
    为10°
96         error = 0
97         start_idx = 7 # 区间可以动态调整, 防止边缘峰干扰,
    增大误差
98         end_idx = len(peak_wavelengths) - 2
99
100        for i in range(start_idx, end_idx):
101            λ1, λ2 = peak_wavelengths[i], peak_wavelengths[i
    +1]
102            n1_sq = a - b * λ1**2
103            n2_sq = a - b * λ2**2
104
105            # 计算根号下的式子(加入判断确保根号下非负)
106            term1 = math.sqrt(n1_sq - math.sin(theta0_rad)**2)
107            if n1_sq > math.sin(theta0_rad)**2 else 0
108            term2 = math.sqrt(n2_sq - math.sin(theta0_rad)**2)
109            if n2_sq > math.sin(theta0_rad)**2 else 0
110
111            if term1 == 0 or term2 == 0:
112                return float('inf') # 没有物理意义, 进行惩罚
    程序
113
114            # 核心公式: 理论和实际的级次差都为1
115            left = 2 * d * term2 / λ2

```

```

114         right = 2 * d * term1 / λ1
115         theory_diff = left - right
116         actual_diff = 1
117         error += (theory_diff - actual_diff) **2 # 对平方
误差累计求和
118         return error
119
120     # 目标函数2：增强对折射率约束(用指数级惩罚来确保物理合理性)
121     def objective2(params):
122         a, b, _ = params
123         penalty = 0
124         for λ in peak_wavelengths:
125             n_sq = a - b * λ**2
126             if n_sq < 1.0**2: # 检测折射率平方是否小于1(不符合实际就要约束)
127                 # 指数级惩罚：n越小，惩罚增长越快
128                 penalty += 100 * np.exp(10 * (1 - n_sq))
129             if n_sq < 0: # 避免开方为负
130                 penalty += 1e15 # 更严重的惩罚
131         return penalty
132
133     # 多目标优化（增强约束权重）
134     def multi_objective(params):
135         return objective1(params) + 0.5 * objective2(params)
136
137     # 约束权重设定为0.5
138
139     # 多初始点优化
140     initial_guesses = [
141         [3.1, 0.0016, 9.36],
142         [4.9, 0.0030, 10.23],
143         [5.8, 0.0065, 11.81]
144     ]
145
146     bounds = [

```



```

145         (1.5, 6.5),          # 要确保  $a - b\lambda^2 \geq 1$  ( $\lambda$ 最大时,  $a$ 需足
符合物理意义(折射率 $\geq 1$ )
146         (0.0005, 0.008),    # 降低 $b$ 的上限可以减缓 $n^2$ 的下降速度
147         (9.0, 12.0 )        # 厚度 $d$ 的范围
148     ]
149
150     # 筛选最优结果
151     best_result = None
152     best_fun = float('inf')
153     for guess in initial_guesses:
154         res = minimize(
155             multi_objective,
156             guess,
157             bounds=bounds,
158             method='SLSQP',    #选择序列最小二乘规划解决带约束
非线性优化问题
159             options={'maxiter': 5000, 'ftol': 1e-10, 'eps': 1e
-6}
160         )
161         if res.success and res.fun < best_fun:
162             best_fun = res.fun
163             best_result = res
164
165     if best_result is None:
166         raise ValueError("所有初始点优化均失败, 请检查参数范围
或数据")
167
168     optimal_params = {          #使用字典存储待优化参数
169         'a': best_result.x[0],
170         'b': best_result.x[1],
171         'd': best_result.x[2]
172     }
173
174     # 优化后验证折射率合理性 + 可视化折射率分布
175     n_values = [math.sqrt(optimal_params['a'] - optimal_params

```

```

176     ['b'] * λ**2) for λ in peak_wavelengths]
177     print("\n折射率范围验证:")
178     print(f"最小折射率: {min(n_values):.4f}")
179     print(f"最大折射率: {max(n_values):.4f}")
180     if any(n < 1.0 for n in n_values):
181         print("警告: 仍存在折射率<1的不合理值,需进一步调整!")
182     else:
183         print("折射率全部≥1,物理合理!")
184
185     # 优化后验证折射率合理性
186     n_values = [math.sqrt(optimal_params['a'] - optimal_params
187     ['b'] * λ**2) for λ in peak_wavelengths] # 补全平方
188     print("\n折射率范围验证:")
189     print(f"最小折射率: {min(n_values):.4f}")
190     print(f"最大折射率: {max(n_values):.4f}")
191     if any(n < 1.0 for n in n_values):
192         print("警告: 存在折射率<1的不合理值,建议调整参数范围!")
193 )
194
195 # 输出优化结果
196 print("\n优化结果:")
197 print(f"最优参数 a: {optimal_params['a']:.4f}")
198 print(f"最优参数 b: {optimal_params['b']:.6f}")
199 print(f"最优外延层厚度 d: {optimal_params['d']:.4f} μm")
200 print(f"优化状态: {'成功' if best_result.success else '失败'}")
201
202 print(f"目标函数值: {best_result.fun:.4e}")
203
204 # 调用优化函数并输出厚度结果
205 optimal_params = optimize_parameters(attach1_peak_λ,
206     attach1_delta_λ)
207 print(f"\n优化得到的外延层厚度 d: {optimal_params['d']:.4f} μm")
208

```

code2.py

```

1 def optimize_parameters(peak_wavelengths, delta_λ, theta0=10):
2     """优化计算a,b,d参数(采用多初始点优化,同时增强折射率约束)
3     """
4
5     theta0_rad = math.radians(theta0)
6
7     # 待优化的目标函数1: 理论波长差公式
8     def objective1(params):
9         a, b, d = params
10        theta0_rad = math.radians(10) # 按题目要求入射角固定
11        为10°
12        error = 0
13        start_idx = 7 # 区间可以动态调整,防止边缘峰干扰,
14        增大误差
15        end_idx = len(peak_wavelengths) - 2
16
17        for i in range(start_idx, end_idx):
18            λ1, λ2 = peak_wavelengths[i], peak_wavelengths[i
19            +1]
20
21            n1_sq = a - b * λ1**2
22            n2_sq = a - b * λ2**2
23
24            # 计算根号下的式子(加入判断确保根号下非负)
25            term1 = math.sqrt(n1_sq - math.sin(theta0_rad)**2)
26            if n1_sq > math.sin(theta0_rad)**2 else 0
27            term2 = math.sqrt(n2_sq - math.sin(theta0_rad)**2)
28            if n2_sq > math.sin(theta0_rad)**2 else 0
29
30            if term1 == 0 or term2 == 0:
31                return float('inf') # 没有物理意义,进行惩罚
32
33            # 核心公式:理论和实际的级次差都为1
34            left = 2 * d * term2 / λ2
35            right = 2 * d * term1 / λ1
36            theory_diff = left - right

```

```

29         actual_diff = 1
30         error += (theory_diff - actual_diff) **2 # 对平方
误差累计求和
31     return error
32
33     # 目标函数2：增强对折射率约束(用指数级惩罚来确保物理合理性)
34     def objective2(params):
35         a, b, _ = params
36         penalty = 0
37         for λ in peak_wavelengths:
38             n_sq = a - b * λ**2
39             if n_sq < 1.0**2: # 检测折射率平方是否小于1(不符合实际就要约束)
40                 # 指数级惩罚：n越小，惩罚增长越快
41                 penalty += 100 * np.exp(10 * (1 - n_sq))
42             if n_sq < 0: # 避免开方为负
43                 penalty += 1e15 # 更严重的惩罚
44         return penalty
45
46     # 多目标优化（增强约束权重）
47     def multi_objective(params):
48         return objective1(params) + 0.5 * objective2(params)
49
# 约束权重设定为0.5
50
51     # 筛选最优结果
52     best_result = None
53     best_fun = float('inf')
54     for guess in initial_guesses:
55         res = minimize(
56             multi_objective,
57             guess,
58             bounds=bounds,
59             method='SLSQP', #选择序列最小二乘规划解决带约束
非线性优化问题

```

```

59         options={'maxiter': 5000, 'ftol': 1e-10, 'eps': 1e
60         -6}
61     )
62     if res.success and res.fun < best_fun:
63         best_fun = res.fun
64         best_result = res
65
66
67 def sensitivity_analysis(optimal_params, peak_wavelengths,
68     delta_λ, theta0=10):
69     """偏导数法的灵敏度分析"""
70     a, b, d = optimal_params['a'], optimal_params['b'],
71     optimal_params['d']
72     theta0_rad = math.radians(theta0)
73     h = 1e-6 # 微小的扰动步长
74
75     print("\n=== 偏导法参数灵敏度分析 ===")
76
77     # 计算原始误差
78     original_error = calculate_error(a, b, d, peak_wavelengths
79     , delta_λ, theta0_rad)
80
81     # 计算a的偏导数
82     error_a_plus = calculate_error(a + h, b, d,
83     peak_wavelengths, delta_λ, theta0_rad)
84     a_partial_deriv = (error_a_plus - original_error) / h
85
86     # 计算b的偏导数
87     error_b_plus = calculate_error(a, b + h, d,
88     peak_wavelengths, delta_λ, theta0_rad)
89     b_partial_deriv = (error_b_plus - original_error) / h
90
91     # 计算d的偏导数
92     error_d_plus = calculate_error(a, b, d + h,

```

```

peak_wavelengths, delta_λ, theta0_rad)
88     d_partial_deriv = (error_d_plus - original_error) / h
89
90     # 归一化处理
91     norm_partial_a = abs(a_partial_deriv * a / original_error)
92     norm_partial_b = abs(b_partial_deriv * b / original_error)
93     norm_partial_d = abs(d_partial_deriv * d / original_error)
94
95     print("\n参数灵敏度(偏导):")
96     print(f"参数a的偏导数: {a_partial_deriv:.4e}")
97     print(f"参数b的偏导数: {b_partial_deriv:.4e}")
98     print(f"参数d的偏导数: {d_partial_deriv:.4e}")
99
100    print("\n归一化灵敏度(弹性系数):")
101    print(f"参数a的弹性系数: {norm_partial_a:.4f}")
102    print(f"参数b的弹性系数: {norm_partial_b:.4f}")
103    print(f"参数d的弹性系数: {norm_partial_d:.4f}")
104
105    def calculate_error(a, b, d, peak_wavelengths, delta_λ,
theta0_rad):
106        """计算固定参数下的累计误差"""
107        error = 0        #误差先初始化
108        start_idx = 7
109        end_idx = len(peak_wavelengths) - 2
110
111        for i in range(start_idx, end_idx):
112            λ1, λ2 = peak_wavelengths[i], peak_wavelengths[i+1]
113            n1_sq = a - b * λ1**2
114            n2_sq = a - b * λ2**2
115
116            term1 = math.sqrt(n1_sq - math.sin(theta0_rad)**2) if
n1_sq > math.sin(theta0_rad)**2 else 0
117            term2 = math.sqrt(n2_sq - math.sin(theta0_rad)**2) if
n2_sq > math.sin(theta0_rad)**2 else 0
118

```

```

119         if term1 == 0 or term2 == 0:
120             return float('inf')
121
122         left = 2 * d * term2 / λ2
123         right = 2 * d * term1 / λ1
124         theory_diff = left - right
125         error += (theory_diff - 1) ** 2
126
127     return error
128
129 # 执行优化和灵敏度分析
130 if __name__ == "__main__":
131     result, params = optimize_parameters(attach1_peak_λ,
132                                         attach1_delta_λ)
133
134     sensitivity_analysis(params, attach1_peak_λ,
135                         attach1_delta_λ)

```

code3.py

```

1 def load_and_preprocess(spectral_data_path):
2     """数据加载函数"""
3     df = pd.read_excel(spectral_data_path, header=None, names
4                       =["v", "R"], engine='openpyxl')
5     df = df.apply(pd.to_numeric, errors='coerce').dropna()
6     df = df.sort_values("v").reset_index(drop=True)
7     # 将百分比的反射率数据转换为小数
8     df["R"] = df["R"] / 100.0
9
10    # 使用Savitzky-Golay滤波器进行数据平滑处理
11    from scipy.signal import savgol_filter
12    window_size = 21      # 窗口大小(奇数)
13    poly_order = 3        # 多项式阶数
14    df["R"] = savgol_filter(df["R"], window_size, poly_order)
15
16    df["λ"] = 10**4 / df["v"]

```

```

16     return df, df["v"].values, df["R"].values, df["λ"].values
17
18 def calculate_fresnel_coefficients(n1, n2, theta0=10):
19     """菲涅耳系数计算"""
20     theta0_rad = math.radians(theta0)
21     sin_theta1 = math.sin(theta0_rad) / n1
22     cos_theta1 = cmath.sqrt(1 - sin_theta1**2) if abs(1 -
sin_theta1**2) > 0 else 0
23
24     r1s = (math.cos(theta0_rad) - n1 * cos_theta1) / (math.cos
(theta0_rad) + n1 * cos_theta1)
25     r1p = (n1 * math.cos(theta0_rad) - cos_theta1) / (n1 *
math.cos(theta0_rad) + cos_theta1)
26     r1 = cmath.sqrt((r1s**2 + r1p**2)/2)
27
28     sin_theta2 = math.sin(theta0_rad) / n2
29     cos_theta2 = cmath.sqrt(1 - sin_theta2**2) if abs(1 -
sin_theta2**2) > 0 else 0
30
31     r2s = (n1 * cos_theta1 - n2 * cos_theta2) / (n1 *
cos_theta1 + n2 * cos_theta2)
32     r2p = (n2 * cos_theta1 - n1 * cos_theta2) / (n2 *
cos_theta1 + n1 * cos_theta2)
33     r2 = cmath.sqrt((r2s**2 + r2p**2)/2)
34
35     return r1, r2
36
37 class SiCOpticalModel:
38     """构建SiC的光学模型"""
39     def __init__(self, m_eff, N, omega_p, a=6.52):    #定义相
关光学参数
40         self.a = a
41         self.m_eff = m_eff
42         self.N = N
43         self.omega_p = omega_p

```



```

44
45     def set_carrier_params(self, N, m_eff, mu=1): #规定载流子
浓度 为1
46         e = 4.8e-10
47         self.omega_p = math.sqrt(N * e**2 / 8.85e-12 * m_eff)
48         self.gamma = e / (m_eff * mu)
49
50     def dielectric_function(self, wavelength):
51         wavenumber = 1e4 / wavelength #波数与波长的相互转
换
52         omega = 2 * math.pi * 3e14 * wavenumber
53         eps_carrier = -self.omega_p**2 / (omega * (omega + 1j*
self.gamma)) if self.omega_p > 0 else 0
54         return self.a * (1 + eps_carrier)
55
56 def global_fit_refractive_index(measured_R, wavelengths,
theta0=10):
57     """全局拟合函数，添加进度显示"""
58     bounds = [
59         (3.0, 6.0), # 厚度d的范围(μm)
60         (5e16, 5e17), # 载流子浓度范围(cm-3)
61         (1.82e-31, 6.83e-31), # 有效质量范围
62         (3, 10), # 衬底折射率
63         (6.5, 6.9) # 参数a的范围
64     ]
65
66     model = SiCOpticalModel(m_eff=1.0, N=5e17, omega_p=0)
#先对模型进行变量初始化
67
68     res = dual_annealing( #双退火算法可以一边模拟退火，一边
局部搜索，避免局部最优
69         objective,
70         bounds=bounds,
71         maxiter=200,
72         initial_temp=3000,

```

```

73         visit=2.5,
74         accept=-5.0,
75         no_local_search=False
76     )
77
78     # 求出最优结果
79     d_opt, N_opt, m_eff_opt, n2_opt, a_opt = res.x
80     model.set_carrier_params(N=N_opt, m_eff=m_eff_opt)
81     model.a = a_opt
82
83     n1_values = []
84     for  $\lambda$  in wavelengths:
85         eps = model.dielectric_function( $\lambda$ )
86         n1_values.append(cmath.sqrt(eps))
87
88     return {
89         'n1': n1_values,
90         'n2': n2_opt,
91         'd': d_opt,
92         'N': N_opt,
93         'm_eff': m_eff_opt,
94         'a': a_opt,
95         'success': res.success,
96         'message': res.message,
97         'rmse': math.sqrt(abs(res.fun))    #取绝对值目的是确保
平方根计算有理化
98     }
99
100 def calculate_reflectivity(n1, n2, d, wavelength, theta0=10):
101     """反射率计算"""
102     n1_complex = complex(n1) if not isinstance(n1, complex)
103     else n1
104     n2_complex = complex(n2) if not isinstance(n2, complex)
105     else n2

```

```

105     r1, r2 = calculate_fresnel_coefficients(n1_complex,
106     n2_complex, theta0)
107     delta = calculate_phase_difference(n1_complex, d,
108     wavelength, theta0)
109
110     numerator = (r1**2 + r2**2) + 2 * r2 * r1 * np.cos(cmath.
111     exp(1j * delta))
112     denominator = 1 + 2 * r2 * r1 * np.cos(cmath.exp(1j *
113     delta)) + (r1 * r2)**2
114     return (numerator / denominator)
115
116 def calculate_phase_difference(n1, d, wavelength, theta0=10):
117     """相位差计算"""
118     theta0_rad = math.radians(theta0)
119     sin_theta1 = math.sin(theta0_rad) / n1
120     cos_theta1 = cmath.sqrt(1 - sin_theta1**2) if abs(1 -
121     sin_theta1**2) > 0 else 0
122     delta = (4 * cmath.pi * n1 * d * cos_theta1) / wavelength
123     + cmath.pi
124     return delta
125
126 if __name__ == "__main__":
127     # 将附件数据导入
128     df, v, R, λ = load_and_preprocess("附件3.xlsx")
129
130     print("开始进行优化计算...")
131     result = global_fit_refractive_index(R, λ)
132
133     print("\n最终拟合结果:")
134     print(f"衬底折射率 n2: {result['n2']:.4f}")
135     print(f"外延层厚度 d: {result['d']:.4f} μm")
136     print(f"载流子浓度 N: {result['N']:.2e} cm3")
137     print(f"有效质量 m_eff: {result['m_eff']:.4f}")
138     print(f"拟合RMSE: {result['rmse']:.6f}")
139     print(f"优化状态: {'成功' if result['success'] else '失败'")

```

```
'} - {result['message']}")
```